COMP4141 Slides Week 7

Brendan Mabbutt

UNSW

March 31, 2026

Admin

Feedback

- Problem points from my marking over the week were the pumping theorem for context-free/regular languages
- **Stars are only for feedback**, not for marks

Many-One Reductions

Decision Problems and Decidability

Decision Problems

Decision problems are problems for which the answer is either **Yes** or **No**.
A decision problem is *decidable* (or *recursive*) if it is accepted by a decider: for every Yes instance, the machine accepts in finite time, and for every No instance, the machine rejects in finite time.

Recursively Enumerable

RE

A language L is RE (recursively enumerable, or Turing-recognisable) if there exists a Turing machine that accepts L.

Enumerator

An enumerator is a Turing machine-like construct that prints an (infinite) list of strings onto the tape.

Enumerator Characterisation

A language is Turing-recognisable if and only if some enumerator enumerates it.

Using Enumerator

We can solve this problem from last week with Enumerators.

Problem 5 (Week 5)

Show that the language $\{\langle M \rangle \mid M \text{ is a Turing machine with } L(M) \neq \emptyset\}$ is Turing-recognisable (recursively enumerable).

An Enumerator:

$T =$ "Repeat for $i = 1, 2, \ldots$:

1. Take the first i words $w_1, w_2, \ldots, w_i$.
2. Take the first i Turing machines $M_1, M_2, \ldots, M_i$.
2. Run each Turing machine on each word for i steps
3. Print out any Turing machines M that accept and have not been printed yet."

Undecidability

coRE

A decision problem is coRE if its complement is recursively enumerable.

Undecidable Languages

There exist undecidable languages, as well as languages that are not in RE.

Relationship

Recall that the class of decidable languages (Rec) satisfies

$$\text{Rec} = \text{RE} \cap \text{coRE}.$$

Halting Problem

Acceptance Problem

The acceptance problem for Turing machines,

$$A_{TM} = \{\langle M, w\rangle \mid M \text{ is a TM that accepts } w\},$$

is RE but undecidable.

Halting Problem

$$H_{TM} = \{\langle M, w\rangle \mid M \text{ halts on input } w\}$$

is undecidable.

Proving Undecidability; Diagonlisation

recall how we proved

Problem 4

An *infinite word* over Σ is an infinite sequence $\rho = a_0 a_1 a_2 \ldots$ such that $a_i \in \Sigma$ for all i. Show that the set of infinite words over $\{0, 1\}$ is uncountable.

is undecidable last week; we can use a similar idea to show the following is undecidable.

Acceptance Problem

The acceptance problem for Turing machines,

$$A_{TM} = \{\langle M, w\rangle \mid M \text{ is a TM that accepts } w\},$$

is RE but undecidable.

Proving Undecidability; Diagonalisation

The idea is that if A_{TM} were decidable, we would be able to compute the following table.

	w_1	w_2	w_3	$\dots$
M_1	Accept	Accept	Reject	
M_2	Reject	Accept	Reject	
M_3	Accept	Reject	Reject	
$\vdots$				$\ddots$

Table: Computation table for A_{TM}

But then we could construct a Turing machine M that rejects w_1, rejects w_2, and accepts w_3, so it cannot be any of the enumerated Turing machines; a contradiction.

Reductions

Many-One Reduction

A language A is many-one reducible to a language B, written $A \leq_m B$, if there exists a computable function f such that for every w,

$$w \in A \iff f(w) \in B.$$

The function f is called a reduction from A to B.

Using Reductions

Theorem

We have the following

- If $A \leq_m B$ and B is decidable, then A is decidable.
- If $A \leq_m B$ and A is undecidable, then B is undecidable.

A Non-Pathological / Non-Esoteric Example of an Undecidable Problem

See page 227 of Sipser: *A Simple Undecidable Problem*.

Recap: Other Types of Turing Machines

Other Turing Machine Variations

Other variations include:

- Multitape TM: each tape has its own head for reading and writing. Initially, the input appears on tape 1, and the others start out blank.
- Doubly infinite tape
- Left-bounded tape
- Nondeterministic TMs.

Equivalence of Turing Machine Variations

All the above Turing machine variations are equivalent in the languages they can represent.

Rice's Theorem

Semantic Properties

Let $P \subseteq \{\langle M \rangle \mid M \text{ is a TM}\}$ be a language of Turing machine descriptions.
P is *nontrivial* if it contains some but not all Turing machine descriptions.
P is *semantic* if whenever $L(M_1) = L(M_2)$, we have

$$M_1 \in P \iff M_2 \in P.$$

Rice's Theorem

Every nontrivial semantic property of Turing machine languages is undecidable.

Big O Notation

Big O

Big O

Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We say that $f(n) = O(g(n))$ if there exist constants $c, n_0 > 0$ such that for all $n \geq n_0$,

$$f(n) \leq c \cdot g(n).$$

Equivalently,

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} < \infty.$$

In this case, g is an asymptotic upper bound for f.

Little o

Little o

Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We say that $f(n) = o(g(n))$ if

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0.$$

That is, for any $c > 0$, there exists $n_0 > 0$ such that

$$f(n) < c \cdot g(n) \quad \text{for all } n \geq n_0.$$

Time Complexity

Time Complexity

Let $t : \mathbb{N} \to \mathbb{R}_{\geq 0}$. Define the time complexity class $\mathrm{TIME}(t(n))$ to be the collection of all languages that are decidable by a Turing machine running in $O(t(n))$ time.

The type of Turing machine matters for this!

(For problem 8) Improving time complexity

Consider the language $\{0^i 1^i : i \in \mathbb{N}\}$. Then:

- First, check that the string consists of a single block of 0s followed by a single block of 1s, and that the total length is even.
- One can decide the language in $O(n^2)$ time by scanning back and forth, crossing out pairs of 0s and 1s each time.
- Alternatively, in $O(n \log n)$ (which is $o(n^2)$), scan back and forth, removing every second 0 and 1, and reject if one group is left while the other isn't. This halves each group at every step, so equality of the counts is necessary and sufficient.
- Alternatively, in $O(n)$ we can do this using a 2-tape TM by copying the 0s onto one tape and then comparing them with the 1s.

W7 P1

Problem 1

Show that $\leq_m$ is transitive, i.e., if $A \leq_m B$ and $B \leq_m C$ then $A \leq_m C$.

W7 P2

Problem 2

Show that the language

$$\{\langle M \rangle \mid M \text{ is a Turing machine such that } L(M) \text{ is an infinite language}\}$$

is undecidable.

W7 P3

Problem 3

Show that a language A is decidable iff $A \leq_m 0^*1^*$.

W7 P4

Problem 4

Consider the problem of determining whether a two-tape Turing machine ever writes a non-blank symbol on its second tape when it is run on input w. Formulate this problem as a language and show that it is undecidable.

W7 P5

Problem 5

Using the definitions of O and o, determine which of the following assertions are true, carefully justifying your answer.

(a) $10^{100} \cdot n = O(\frac{1}{10^{100}} \cdot n)$

(b) $\frac{1}{10^{100}} \cdot n = o(10^{100} \cdot n)$

(c) $257n^3 + 3n^2 + 28 = O(n^3)$

(d) $257n^3 + 3n^2 + 28 = o(10^{100} \cdot n^3)$

(e) $n \log n = o(n^2)$

(f) $n^{100} = o(2^n)$

W7 P6

Problem 6

Prove using the definitions that $f = o(g)$ implies $f = O(g)$.

W7 P7

Problem 7

Suppose f and g are strictly positive (always greater than zero) functions, with $g = O(f)$. Is it possible that $f = o(g)$? (Prove or disprove.)

W7 P8

Problem 8

Consider the following language:

$$\{u\#v\#w \ : \ u, v, w \in \{0,1\}^* \text{ are binary numbers such that } w = u + v\}$$

(a) Characterize the time complexity of this language on a (one tape) Turing machine.

(b) Is it possible to do better on a two-tape Turing machine?

(c) Compare this to the time complexity for a program you could write on a standard computer with, say, 10 registers that can hold fixed length words, and random access to memory, i.e., a machine with an instruction for fetching the word at a given memory location into one of the registers. Assume that the string $u\#v\#w$ is located at a known starting location in memory.

Problem 8

Consider the following language:

$$\{u\#v\#w \ : \ u, v, w \in \{0,1\}^* \text{ are binary numbers such that } w = u + v\}$$

(d) What would be the time complexity on a standard computer if the input is too large to be written in main memory, and were instead written out on a very long tape, with just a single read head? This model is relevant for very large databases. Problems like processing information from the World Wide Web, such as Google's construction of the index for the web that it uses to answer your search queries, need to deal with a similar difficulty of not being able to store everything in main memory, though fetching data from the internet is somewhat different from reading a tape, and a bit more like having a large external random access memory with a large latency.